```
!pip install langchain chromadb sentence-transformers openai langchain-community langchain-openai
```

```
etes>=28.1.0->chromadb) (1.17.0)
(from kubernetes>=28.1.0->chromadb) (2.9.0.post0)
lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.9.0)
 kubernetes>=28.1.0->chromadb) (2.0.0)
 (from kubernetes>=28.1.0->chromadb) (2.5.0)
ubernetes>=28.1.0->chromadb) (0.10)
2/dist-packages (from langchain-classic<2.0.0,>=1.0.0->langchain-community) (1.1.1)
s (from langchain-core<2.0.0,>=1.2.10->langchain) (1.33)
 (from langchain-core<2.0.0,>=1.2.10->langchain) (0.14.1)
st-packages (from langgraph<1.2.0,>=1.1.1->langchain) (4.0.1)
-packages (from langgraph<1.2.0,>=1.1.1->langchain) (1.0.8)
ages (from langgraph<1.2.0,>=1.1.1->langchain) (0.3.12)
ggraph<1.2.0,>=1.1.1->langchain) (3.6.0)
s (from langsmith<1.0.0,>=0.1.125->langchain-community) (1.0.0)
 langsmith<1.0.0,>=0.1.125->langchain-community) (0.25.0)
untime>=1.14.1->chromadb) (25.12.19)
ime>=1.14.1->chromadb) (5.29.6)
>=1.14.1->chromadb) (1.14.0)
ackages (from opentelemetry-api>=1.2.0->chromadb) (8.7.1)
ackages (from opentelemetry-exporter-otlp-proto-grpc>=1.2.0->chromadb) (1.73.1)
/python3.12/dist-packages (from opentelemetry-exporter-otlp-proto-grpc>=1.2.0->chromadb) (1.40.0)
ages (from opentelemetry-exporter-otlp-proto-grpc>=1.2.0->chromadb) (1.40.0)
n3.12/dist-packages (from opentelemetry-sdk>=1.2.0->chromadb) (0.61b0)
(from pydantic<3.0.0,>=2.7.4->langchain) (0.7.0)
from pydantic<3.0.0,>=2.7.4->langchain) (2.41.4)
s (from pydantic<3.0.0,>=2.7.4->langchain) (0.4.2)
from pydantic-settings>=2.0->chromadb) (1.2.2)
s (from requests<3.0.0,>=2.32.5->langchain-community) (3.4.6)
from rich>=10.11.0->chromadb) (4.0.0)
 (from rich>=10.11.0->chromadb) (2.20.0)
chemy<3.0.0,>=1.4.0->langchain-community) (3.3.2)
tiktoken<1.0.0,>=0.7.0->langchain-openai) (2025.11.3)
=1.11.0->sentence-transformers) (75.2.0)
orch>=1.11.0->sentence-transformers) (3.6.1)
1.0->sentence-transformers) (3.1.6)
ormers<6.0.0,>=4.41.0->sentence-transformers) (0.24.0)
n transformers<6.0.0,>=4.41.0->sentence-transformers) (0.7.0)
r>=0.9.0->chromadb) (8.3.1)
n typer>=0.9.0->chromadb) (1.5.4)
rom typer>=0.9.0->chromadb) (0.0.4)
uvicorn[standard]>=0.18.3->chromadb) (0.7.1)
icorn[standard]>=0.18.3->chromadb) (0.22.1)
uvicorn[standard]>=0.18.3->chromadb) (1.1.1)
uvicorn[standard]>=0.18.3->chromadb) (15.0.1)
cit-learn->sentence-transformers) (1.5.3)
rom scikit-learn->sentence-transformers) (3.6.0)
lib-metadata<8.8.0,>=6.0->opentelemetry-api>=1.2.0->chromadb) (3.23.0)
jsonpatch<2.0.0,>=1.33.0->langchain-core<2.0.0,>=1.2.10->langchain) (3.1.1)
 langgraph-checkpoint<5.0.0,>=2.1.0->langgraph<1.2.0,>=1.1.1->langchain) (1.12.2)
wn-it-py>=2.2.0->rich>=10.11.0->chromadb) (0.1.2)
n sympy->onnxruntime>=1.14.1->chromadb) (1.3.0)
(from typing-inspect<1,>=0.4.0->dataclasses-json<0.7.0,>=0.6.7->langchain-community) (1.1.0)
inja2->torch>=1.11.0->sentence-transformers) (3.0.3)
equests-oauthlib->kubernetes>=28.1.0->chromadb) (3.3.1)
```

```
from langchain_huggingface.embeddings import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

print("Embedding model loaded successfully!")
```

```
Loading weights: 100%                    103/103 [00:00<00:00, 456.71it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expe
Embedding model loaded successfully!
```

```
from langchain_community.vectorstores import Chroma

documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation.",
    "Supervised learning uses labeled data.",
    "Unsupervised learning finds hidden patterns.",
    "Reinforcement learning learns through rewards.",
    "Transformers are widely used in NLP tasks.",
    "Large Language Models generate human-like text."
]

# Create Vector Database
db = Chroma.from_texts(documents, embedding_model)

print("Vector Database created successfully!")
```

```
Vector Database created successfully!
```

```
retriever = db.as_retriever()

print("Retriever ready!")
```

```
Retriever ready!
```

```
retriever = db.as_retriever(search_kwargs={"k": 3})
```

```
import os

os.environ["OPENAI_API_KEY"] = "your_api_key_here"
```

```
from langchain_openai.llms import OpenAI

llm = OpenAI(temperature=0)

print("LLM initialized!")
```

```
LLM initialized!
```

```
custom_docs = [
    "Healthcare AI helps in disease prediction.",
    "Finance AI is used for fraud detection.",
    "AI chatbots improve customer service."
]

db = Chroma.from_texts(custom_docs, embedding_model)
retriever = db.as_retriever()

print("Custom dataset loaded!")
```

```
Custom dataset loaded!
```